



Metodología de la Programación

Grado en Ingeniería Informática

Curso 2025/2026

Torrejón Vera, Daniel

Trujillo Añon, Pablo

López Hidalgo, Mario

Proyecto Modularidad

Escape Room: ESI

Índice general

Índice general	1
1. Introducción	2
2. Documentación de usuario	3
2.1. Descripción funcional	3
2.2. Tecnología	4
2.3. Manual de instalación	4
2.4. Acceso al sistema	6
2.5. Manual de referencia	7
3. Documentación de sistema	9
4. Especificación del sistema	9
4.1. Análisis y especificación de requisitos	9
4.2. Descomposición modular	10
4.3. Plan de desarrollo del software	10
4.4. Módulos	11
4.5. Plan de prueba	12
4.6. Plan de pruebas de integración	22
4.7. Plan de pruebas de aceptación	22
Referencias	25

1. Introducción

Escape Room: ESI es un juego de aventura conversacional basado en texto donde el jugador asume el rol de un estudiante atrapado en la Escuela Superior de Ingeniería (ESI) de la UCA. El objetivo es explorar las instalaciones, interactuar con objetos y resolver acertijos para lograr llegar al laboratorio,.

El proyecto ha sido desarrollado siguiendo un diseño modular en lenguaje C, dividiendo la lógica del juego en componentes independientes (como la gestión de mapas, el inventario y el sistema de usuarios). Esta estructura facilita el mantenimiento del código y permite entender el funcionamiento del programa de forma clara y organizada.

2. Documentación de usuario

2.1. Descripción funcional

El propósito de **“Misión ESI”** es ofrecer una experiencia de juego donde el usuario debe gestionar sus recursos y conocimientos para escapar de la Escuela. El sistema se basa en las siguientes funcionalidades principales:

Sistema de Gestión de Usuarios

- Registro: El sistema permite crear una cuenta nueva solicitando un nombre de usuario y una contraseña segura. Estos datos se almacenan de forma persistente.
- Autenticación: Para comenzar una partida, el usuario debe iniciar sesión con sus credenciales. Esto permite que el juego identifique al alumno y pueda (en versiones futuras) guardar su progreso.

Mecánicas de Juego (¿Qué se puede hacer?)

- Exploración de la ESI: El jugador puede moverse entre distintas estancias reales del centro (Entrada, Cafetería, Biblioteca, Laboratorios).
- Interacción con el entorno: Se pueden examinar las salas para obtener descripciones detalladas que esconden pistas.
- Gestión de Inventario: El usuario puede recoger objetos clave (como el carnet de la UCA o una memoria USB) y consultarlos en cualquier momento.
- Resolución de Puzzles: El sistema permite introducir comandos para intentar resolver acertijos (por ejemplo, introducir una clave numérica en un teclado electrónico para abrir un laboratorio).

Limitaciones del Sistema (¿Qué NO hace?)

- El juego es puramente textual; no incluye gráficos ni sonido.
- No permite el modo multijugador ni interacción entre diferentes usuarios registrados.
- No se puede volver atrás una vez que se ha tomado una decisión crítica que lleva al fin de la partida (Game Over).

Objetivo Final

La partida finaliza con éxito cuando el usuario logra desbloquear la puerta del laboratorio tras haber resuelto los puzzles obligatorios de cada zona.

2.2. Tecnología

En esta sección se describen las herramientas y tecnologías utilizadas para el desarrollo de **Misión ESI**. El proyecto se ha centrado en el uso de estándares abiertos y herramientas compatibles con los laboratorios de la Escuela Superior de Ingeniería.

- **Lenguaje de programación:** C (Estándar C11). Se ha elegido por su eficiencia y por ser el lenguaje base para el aprendizaje de la gestión de memoria y modularidad en la asignatura.
- **Compilador:** GCC (GNU Compiler Collection). Utilizado con los flags de depuración `-Wall` y `-Wextra` para asegurar un código limpio y sin errores lógicos.
- **Entorno de desarrollo (IDE):** Visual Studio Code. Se han empleado estos entornos por su versatilidad y capacidad para gestionar proyectos multimodulares.
- **Sistema Operativo:** Desarrollo realizado en entornos basados en Unix (Linux) y Windows 10/11, garantizando la portabilidad del ejecutable.
- **Librerías estándar:** Uso de `stdio.h` (entrada/salida), `stdlib.h` (gestión de memoria y procesos), `string.h` (manejo de cadenas para comandos y contraseñas)

2.3. Manual de instalación

El sistema ha sido desarrollado íntegramente en lenguaje C estándar, lo que garantiza una alta portabilidad y un consumo de recursos extremadamente bajo. A continuación, se detallan los requisitos del sistema y los pasos necesarios para su correcta instalación y ejecución en el equipo de destino.

Requisitos mínimos de hardware y software

Dado que la aplicación se ejecuta a través de la terminal o línea de comandos y no hace uso de un motor gráfico complejo, los requisitos exigidos al sistema son mínimos, permitiendo su funcionamiento en la práctica totalidad de equipos actuales:

- **Procesador:** 1 GHz o superior (arquitecturas x86, x64 o ARM).

- **Memoria RAM:** 512 MB (el consumo del programa en tiempo de ejecución es del orden de unos pocos megabytes).
- **Almacenamiento:** Al menos 10 MB de espacio libre en disco para albergar los archivos fuente, ficheros de datos (`.txt`) y el archivo ejecutable.
- **Sistema Operativo:** Windows 10/11, cualquier distribución moderna de GNU/Linux, o macOS.
- **Software adicional:** Consola de comandos nativa del sistema. Opcionalmente, un compilador de C (como GCC/MinGW) en caso de querer construir el proyecto desde el código fuente.

Proceso de instalación y configuración

El programa sigue una filosofía portable (stand-alone), por lo que no precisa de un instalador tradicional ni requiere permisos de administrador. Para adecuar el sistema, el usuario debe seguir estos pasos:

1. **Obtención de ficheros:** Descargue el paquete del proyecto comprimido (`.zip`) que contiene el código fuente y el directorio de datos.
2. **Despliegue del entorno:** Extraiga el contenido en una carpeta local de su disco duro. *Nota de configuración:* Es requisito indispensable que los archivos de configuración del entorno (`salas.txt`, `conexiones.txt`, `objetos.txt`, `puzles.txt` y `Jugadores.txt`) se encuentren estrictamente en el mismo directorio raíz que el archivo ejecutable para que el motor de lectura de ficheros pueda localizarlos.
3. **Ejecución directa (Entorno Windows):** Si el paquete incluye el ejecutable precompilado (`juego.exe`), el usuario puede abrir el sistema haciendo doble clic sobre el archivo, o abriendo una terminal en dicho directorio y ejecutando el comando:

```
.\juego.exe
```

4. **Compilación manual (Adecuación a otros SO):** Si se desea ejecutar en un hardware particular o en otro sistema operativo (como Linux), abra una terminal en el directorio del proyecto y utilice el compilador GCC para generar un nuevo binario adaptado a su máquina mediante la instrucción:

```
gcc *.c -o juego
```

Posteriormente, inicie el sistema con `./juego`.

2.4. Acceso al sistema

El acceso y manejo del sistema se ha diseñado para ser intuitivo, interactuando con el usuario exclusivamente a través de texto en la consola de comandos. A continuación, se detallan los procedimientos básicos de entrada, salida y resolución de incidencias.

Inicio de sesión en el sistema

Para arrancar el juego, el usuario debe ejecutar el binario compilado (por ejemplo, mediante `.\juego.exe` o el botón de ejecución de su IDE). Al iniciarse, el sistema presentará una pantalla de bienvenida seguida de un menú de autenticación. Para acceder, el usuario deberá introducir su nombre de jugador por teclado y presionar **Enter**. Si el jugador ya existe en la base de datos (`Jugadores.txt`), el sistema cargará su partida; de lo contrario, se le guiará por un proceso guiado para registrar un nuevo perfil y comenzar la aventura desde la sala inicial.

Salida del sistema

El abandono del juego se puede realizar de forma segura para evitar la pérdida de datos o corrupción de memoria:

- **Salida normal:** Desde el menú principal del juego (o menú de opciones), el usuario debe seleccionar la opción correspondiente a “Salir del juego” introduciendo el número indicado en pantalla. Esto ejecutará las funciones de guardado y de liberación de memoria (como `liberar_datos`) antes de cerrar el programa limpiamente.
- **Salida de emergencia:** Si el usuario necesita cerrar el juego abruptamente en cualquier momento, puede utilizar la combinación de teclas **Ctrl + C** en su teclado. Esto enviará una señal de interrupción a la terminal, cerrando el proceso de inmediato.

Resolución de problemas habituales (Troubleshooting)

A continuación, se listan los problemas más comunes que puede experimentar un jugador y su solución directa:

El programa se cierra de golpe nada más abrirlo: Este es el error más frecuente. Ocurre cuando el ejecutable no encuentra los ficheros de texto con los datos del juego. **Solución:** Asegúrese de que los archivos `salas.txt`, `conexiones.txt`, `objetos.txt`, `puzles.txt` y `Jugadores.txt`

están en la misma carpeta exacta desde donde se está ejecutando el programa.

El juego no reconoce mi usuario al iniciar sesión: El sistema de lectura de consola es sensible a las mayúsculas y espacios en blanco. **Solución:** Compruebe que está escribiendo el nombre exactamente igual que cuando se registró. Si el problema persiste, puede abrir el archivo `Jugadores.txt` con un bloc de notas para verificar cómo está guardado su nombre.

El programa se ha quedado “congelado” o repite texto en bucle (Bucle infinito): Aunque el sistema está testeado, si ocurre un fallo lógico que bloquea la terminal, el usuario perderá el control del teclado. **Solución:** Haga clic dentro de la ventana de la terminal y presione `Ctrl + C` para forzar el cierre. Tras esto, vuelva a ejecutar el programa.

2.5. Manual de referencia

En esta sección se detallan las capacidades funcionales del sistema, los beneficios que aporta su arquitectura, el flujo formal de interacción con el usuario y la gestión de excepciones e informes generados durante su ejecución.

Ventajas del sistema y valor aportado

El diseño modular del Escape Room proporciona una serie de ventajas técnicas y de usabilidad destacables:

- **Alta escalabilidad (Data-Driven):** Al basar la lógica en la lectura de ficheros externos (`.txt`), el sistema permite a los diseñadores añadir nuevas salas, objetos o puzles simplemente editando texto, sin necesidad de reprogramar ni recompilar el código fuente en C.
- **Persistencia y seguridad de progreso:** El sistema guarda automáticamente el avance del usuario, la sala en la que se encuentra y el contenido de su inventario. Esto permite sesiones de juego fragmentadas sin riesgo de pérdida de datos.
- **Gestión eficiente de recursos:** Gracias a funciones como `liberar_datos`, la aplicación asegura un uso óptimo de la memoria dinámica, evitando fugas de memoria (*memory leaks*) incluso en partidas de larga duración.

Uso formal del sistema

Una vez autenticado, el usuario interactúa con el entorno de juego mediante un bucle de eventos basado en menús o comandos de texto. El flujo de uso formal se divide en tres pilares:

1. **Exploración y Navegación:** El sistema lee la estructura de la sala actual y presenta por pantalla la descripción del entorno y las salidas disponibles. El usuario puede seleccionar hacia qué sala conectada desea desplazarse.
2. **Interacción y Resolución:** Si la sala contiene puzzles o bloqueos, el sistema solicitará una acción. El jugador deberá usar la deducción y los objetos correctos para resolver la prueba (por ejemplo, utilizar una "llave" para abrir una puerta concreta).
3. **Gestión de Inventario:** El jugador puede visualizar en cualquier momento los objetos recolectados. El sistema controla de forma dinámica la inserción y eliminación de elementos en la estructura del jugador a medida que se resuelven los puzzles.

Situaciones de error y manejo de excepciones

El sistema está diseñado con un enfoque de programación defensiva para evitar la interrupción abrupta de la partida. Las situaciones de error se clasifican en:

Errores de entrada del usuario: Si el usuario introduce una opción no válida en un menú (ej. una letra cuando se espera un número, o

3. Documentación de sistema

Documentación referida a los aspectos del análisis, diseño, implementación y prueba del *software*, así como la implantación del mismo. En general, debe hacer referencia al sistema, ya que está orientada a los programadores que van a realizar el mantenimiento. Debe estar muy bien estructurada, desde lo más general a lo más interno.

4. Especificación del sistema

En esta sección se aborda la arquitectura técnica del Escape Room, detallando la descomposición del problema original en subproblemas manejables y los módulos diseñados para resolver cada uno de ellos. Además, se describe el plan de desarrollo seguido para asegurar la calidad del software.

4.1. Análisis y especificación de requisitos

El sistema debe cumplir con una serie de necesidades funcionales y técnicas para garantizar una experiencia de juego fluida y robusta:

- **Requisitos Funcionales (RF):**

- **RF1 - Gestión de Entorno:** El sistema debe cargar y representar un mapa de salas interconectadas.
- **RF2 - Control de Inventario:** Debe permitir la recolección, almacenamiento y uso de objetos de forma dinámica.
- **RF3 - Motor de Puzles:** Debe gestionar estados lógicos que bloqueen o desbloqueen el avance según las acciones del jugador.
- **RF4 - Persistencia:** Los datos del jugador (nombre, ubicación e ítems) deben guardarse en ficheros de texto.

- **Requisitos No Funcionales (RNF):**

- **RNF1 - Eficiencia:** El sistema debe liberar toda la memoria dinámica utilizada al finalizar la ejecución.
- **RNF2 - Robustez:** Debe ser capaz de gestionar errores de lectura de ficheros sin provocar un volcado de memoria.

4.2. Descomposición modular

Para abordar la complejidad del juego, el problema se ha descompuesto en seis submódulos especializados que interactúan entre sí:

1. **Módulo de Datos de Entorno (Salas y Conexiones):** Responsable de definir las estructuras de las habitaciones y la topología del mapa (grafo de adyacencia).
2. **Módulo de Objetos e Inventario:** Gestiona la memoria dinámica necesaria para los ítems que el jugador recoge durante la partida.
3. **Módulo de Puzzles:** Controla la lógica de los desafíos, verificando si el jugador posee los requisitos necesarios para superar un obstáculo.
4. **Módulo de Persistencia (Gestión de Ficheros):** Encargado de la entrada/salida de datos desde los archivos `.txt` a las estructuras en memoria RAM.
5. **Módulo de Lógica de Juego (Motor):** Implementa el bucle principal de juego (*Game Loop*), procesando las entradas del usuario y actualizando el estado.
6. **Módulo de Utilidades:** Contiene funciones transversales para la limpieza de pantalla, gestión de cadenas y menús de interfaz.

4.3. Plan de desarrollo del software

El desarrollo se ha estructurado en cuatro fases iterativas para minimizar riesgos:

1. **Fase de Análisis y Diseño de Datos:** Definición de estructuras (`struct`) y punteros necesarios para modelar el juego en C.
2. **Fase de Implementación Modular:** Codificación individual de cada módulo (`.c` y `.h`) siguiendo el principio de ocultación de información.
3. **Fase de Integración:** Unión de los módulos en el programa principal (`main.c`) y resolución de dependencias.
4. **Fase de Pruebas y Validación:** Ejecución de pruebas de caja blanca (Ruta Básica) y caja negra para asegurar la ausencia de fugas de memoria y errores lógicos.

4.4. Módulos

El problema se ha dividido en 9 módulos para maximizar la cohesión y minimizar el acoplamiento entre ellos:

Autenticación: El módulo Autenticador centraliza la lógica de seguridad y administración de usuarios. Sus funciones principales incluyen la lectura/escritura en archivos, la validación de credenciales de acceso y la gestión dinámica de la base de datos de jugadores en tiempo de ejecución.

Jugadores: El módulo jugador es el responsable de la lógica del personaje durante la ejecución del juego. Gestiona de forma dinámica el inventario de objetos, la ubicación del usuario en las distintas salas y garantiza la integridad de los datos del jugador mediante funciones de inicialización y liberación de memoria.

Objetos: El módulo objetos gestiona el catálogo completo de ítems del juego. Su función principal es controlar la ubicación de cada objeto, permitiendo la transferencia de elementos entre el mapa y el inventario del jugador, además de gestionar la persistencia de estos datos en archivos de texto y su visualización por pantalla.

Gestión de ficheros: El módulo gestión de ficheros centraliza las operaciones de entrada/salida y el control del ciclo de vida de los datos. Sus funciones garantizan que el estado global del juego se cargue correctamente desde los archivos de configuración y que todos los recursos se liberen de forma segura al finalizar la ejecución.

Lógica: El módulo lógica define el núcleo del sistema mediante la estructura global `EstadoJuego`. Su propósito es integrar todos los componentes del software (gestión de mapas, inventario, usuarios y puzles) en una única entidad funcional, permitiendo la gestión centralizada de la partida y garantizando la coherencia de los datos durante toda la ejecución.

Puzzles: El módulo puzles administra los desafíos intelectuales del juego. Se encarga de definir los requisitos de victoria para cada acertijo, verificar si el jugador se encuentra en la ubicación adecuada para interactuar con ellos y actualizar el estado de resolución, permitiendo así el avance en la partida o el acceso a nuevas áreas.

Salas: El módulo Salas gestiona la estructura espacial del entorno. Su responsabilidad es definir las propiedades estáticas de cada

ubicación del mapa (nombres, tipos y descripciones), proporcionar las rutinas para mostrar esta información al usuario, y determinar las condiciones de término evaluando si una ubicación específica corresponde al destino final de la partida.

Conexiones: El módulo conexiones administra la red de rutas navegables del juego. Su función principal es definir los enlaces lógicos entre las diferentes salas del mapa y gestionar el control de acceso, evaluando estados de apertura y validando si el jugador cumple los requisitos (posesión de objetos o resolución de puzzles) necesarios para transitar entre ubicaciones.

Utilidades: El módulo utilidades agrupa las herramientas de soporte técnico para la interfaz de usuario. Su función es optimizar la interacción por consola mediante la limpieza de pantalla, la gestión de pausas en la ejecución y la depuración del buffer de entrada, garantizando una navegación fluida y sin errores de lectura de datos.

4.5. Plan de prueba

Debe describir todo el plan de prueba que se ha llevado a cabo. Se distinguen las pruebas realizadas a cada módulo y las pruebas de integración de los distintos módulos probándolo como un todo. Por último, también se debe incluir la descripción del plan de pruebas de aceptación.

Prueba de los módulos

```
//Daniel Torrejon Vera
int anadirObjeto(Jugador *j, char *id_obj) {
    char *aux = (char *)realloc(j->inventario,
                                (j->num_objetos + 1) * sizeof(char *));
    if (aux == NULL) {
        return 0;
    }
    j->inventario = aux;

    j->inventario[j->num_objetos] = (char *)malloc(MAX_ID_OBJ * si
    if (j->inventario[j->num_objetos] == NULL) {
        return 0;
    }
}
```

```

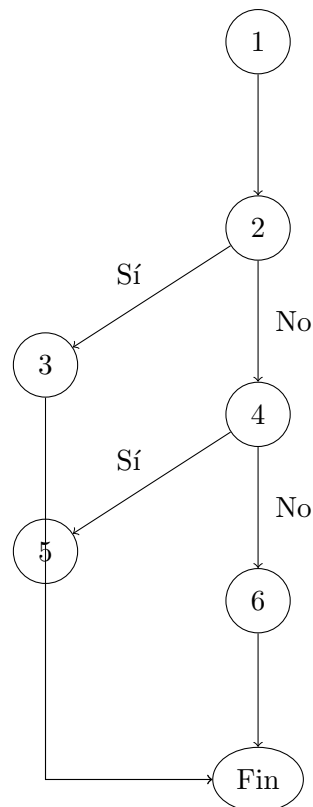
    strncpy(j->inventario[j->num_objetos], id_obj, MAX_ID_OBJ - 1)
    j->inventario[j->num_objetos][MAX_ID_OBJ - 1] = '\0';

    j->num_objetos++;
    return 1;
}

```

La función amplía dinámicamente el inventario del jugador mediante **realloc**. Si la reasignación falla devuelve 0 sin modificar el estado. A continuación reserva memoria para almacenar el identificador del objeto; si este segundo **malloc** falla, también devuelve 0. En caso de éxito, copia el identificador con **strncpy**, fuerza el terminador nulo en la última posición para evitar desbordamientos, incrementa el contador de objetos y devuelve 1.

Grafo de flujo de control (CFG):



Complejidad ciclomática:

- $V(G) = A - N + 2 = 8 - 7 + 2 = 3$
- $V(G) = P + 1 = 2 + 1 = 3$
- $V(G) = regiones = 2(internas) + 1(externa) = 3$

Rutas básicas:

1. $1 \rightarrow 2 \rightarrow 3 \rightarrow Fin$ `realloc` falla, inventario sin modificar, `return 0`
2. $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow Fin$ `realloc` tiene éxito pero `malloc` falla, `return 0`
3. $1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow Fin$ ambas reservas tienen éxito, objeto añadido, `return 1`

Casos de prueba — caja blanca:

- **CT-1** (Ruta 1): se fuerza un fallo de `realloc` simulando memoria agotada. Salida esperada: `return 0`; `j->inventario` y `j->num_objetos` conservan sus valores originales.
- **CT-2** (Ruta 2): `realloc` tiene éxito pero el siguiente `malloc` falla. Salida esperada: `return 0`; el puntero `j->inventario` ya fue actualizado por `aux` pero el nuevo elemento apunta a `NULL` y `num_objetos` no se incrementa.
- **CT-3** (Ruta 3): jugador con `inventario = NULL`, `num_objetos = 0` e `id_obj = ".B01"`; ambas reservas tienen éxito. Salida esperada: `return 1`; `inventario[0]` contiene `".B01"` y `num_objetos == 1`.

Casos de prueba — caja negra:

- **Adición estándar:** jugador con 3 objetos en inventario e `id_obj = ".B04"`. Salida esperada: `return 1`; el inventario pasa a 4 entradas y la última es `".B04"`.
- **ID de longitud máxima:** `id_obj` con exactamente `MAX_ID_OBJ - 1` caracteres. Salida esperada: `return 1`; la cadena se copia completa con terminador nulo en la última posición.
- **ID superior al máximo:** cadena más larga que `MAX_ID_OBJ - 1`. Salida esperada: `return 1`; `strncpy` trunca el ID y el terminador escrito manualmente evita desbordamiento.

- **Fallo de memoria en malloc:** segunda reserva imposible. Salida esperada: `return 0`; el estado del jugador permanece consistente y no se produce acceso a memoria inválida.

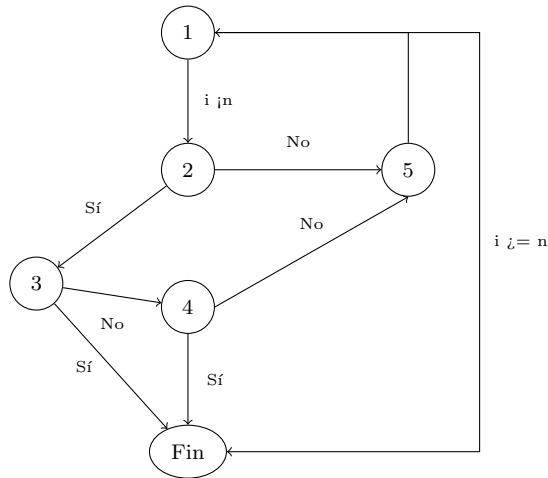
Prueba de los módulos

```
//Mario Lopez Hidalgo
int cogerObjeto(Objeto *lista, int n, char *id_obj, char *id_sala,
int sala_actual = atoi(id_sala_actual);
for (int i = 0; i < n; i++) {
    char id_ingresado[20], id_lista[20];
    strcpy(id_ingresado, id_obj);
    id_ingresado[strcspn(id_ingresado, "\r\n")] = 0;
    strcpy(id_lista, lista[i].id_obj);
    id_lista[strcspn(id_lista, "\r\n")] = 0;

    if (strcmp(id_lista, id_ingresado) == 0) {
        if (atoi(lista[i].localiz) == sala_actual && sala_actu
            strcpy(lista[i].localiz, "Inventario");
            printf("\n[+] Has cogido '%s'!\n", lista[i].noml
            return 1;
        } else if (strcmp(lista[i].localiz, "Inventario") == 0
            printf("\n[X] Ya lo tienes.\n");
            return 0;
        }
    }
}
printf("\n[X] No puedes coger eso.\n");
return 0;
}
```

La función gestiona la transferencia de objetos al inventario del jugador. El proceso incluye la normalización de cadenas mediante `strcspn` para eliminar caracteres de control, la validación de la ubicación física del objeto respecto a la sala del jugador y una comprobación de seguridad para evitar duplicados si el objeto ya consta como "Inventario".

Grafo de flujo de control (CFG):



Complejidad ciclomática:

- $V(G) = A - N + 2 = 9 - 6 + 2 = 5$
- $V(G) = P + 1 = 4 + 1 = 5$ (Puntos de decisión: bucle, ID, sala e inventario).

Rutas básicas:

1. $1 \rightarrow Fin$: El inventario está vacío ($n = 0$).
2. $1 \rightarrow 2 \rightarrow 5 \rightarrow 1$: El ID del objeto no coincide con el buscado.
3. $1 \rightarrow 2 \rightarrow 3 \rightarrow Fin$: El objeto se encuentra y se recoge con éxito.
4. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow Fin$: El ID coincide pero el objeto ya está en el inventario.
5. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1$: El ID coincide pero está en una sala distinta.

Casos de prueba — caja blanca:

- **CT-1** (Ruta 1): Se llama a la función con $n = 0$. *Salida esperada:* `return 0` y mensaje de error.
- **CT-2** (Ruta 2): Se busca un ID inexistente en una lista de 3 objetos. *Salida esperada:* El flujo completa 3 iteraciones y hace `return 0`.
- **CT-3** (Ruta 3): ID ".°BJ01." en la sala "1" siendo el jugador de la sala "1". *Salida esperada:* `return 1` y cambio de localización.

- **CT-4** (Ruta 4): El objeto tiene localización "Inventario". *Salida esperada:* Entra en el bloque `else if`, imprime "Ya lo tienes" y hace `return 0`.
- **CT-5** (Ruta 5): ID correcto pero `lista[i].localiz` es "2" el jugador está en la "1". *Salida esperada:* Salta al nodo 5, continúa el bucle y finalmente retorna 0.

Casos de prueba — caja negra:

- **ID con caracteres de control:** Entrada de `id_obj` con salto de línea (`.espada\n`). *Salida esperada:* `return 1` (la limpieza con `strcspn` debe funcionar).
- **Objeto en sala especial 0:** Se intenta coger un objeto cuya localización es "0". *Salida esperada:* `return 0` (la condición `sala_actual != 0` lo impide).
- **Búsqueda en lista vacía:** Parámetro $n = 0$ con cualquier ID. *Salida esperada:* `return 0` inmediato.

Si en algún punto del documento se quiere hacer referencia a algún documento es necesario incluir la cita donde corresponda, podemos tomar como ejemplo ?.

Prueba de los módulos

//Pablo Trujillo Anon

```
void funcion liberar_datos (E EstadoJuego *juego)
var
    entero: i, j
inicio
    i <- 0, j <- 0                                     (1)

    si (juego->salas != NULL) entonces                 (2)
        liberar(juego->salas)
    fin_si

    si (juego->conexiones != NULL) entonces             (3)
        liberar(juego->conexiones)
    fin_si

    si (juego->objetos != NULL) entonces                (4)
```

```

        liberar(juego->objetos)
    fin_si

    si (juego->puzles != NULL) entonces (5)
        liberar(juego->puzles)
    fin_si

    si (juego->jugadores_bd != NULL) entonces (6)
        liberar(juego->jugadores_bd)
    fin_si

    si (juego->jugador_actual != NULL) entonces (7)
        si (juego->jugador_actual->inventario != NULL) entonces (8)
            desde j <- 0 hasta num_objetos - 1 (9)
                liberar(inventario[j]) (10)
            fin_desde
            liberar(inventario) (11)
        fin_si
        liberar(juego->jugador_actual) (12)
    fin_si (13)

fin_funcion

```

La función `liberar_datos` realiza la liberación ordenada y segura de toda la memoria dinámica asociada al estado del juego. Antes de llamar a `free`, comprueba individualmente que cada puntero no sea `NULL`, evitando así comportamiento indefinido. El bloque más complejo corresponde al jugador actual: si su inventario no es nulo, se recorre en bucle liberando cada elemento almacenado (nodo 10), a continuación se libera el array del inventario (nodo 11) y, por último, la estructura `jugador_actual` (nodo 12). Este orden garantiza que no queden referencias huérfanas ni fugas de memoria.

Grafo de flujo de control (CFG):

Complejidad ciclomática: $V(G) = A - N + 2 = 20 - 13 + 2 = 9$

$V(G) = P + 1 = 8 + 1 = 9$

$V(G) = \text{regiones} = 8 \text{ (internas)} + 1 \text{ (externa)} = 9$

Rutas básicas:

Dado que $V(G) = 9$, el conjunto base está formado por exactamente

9 rutas. Las rutas 1–6 atraviesan los mismos nodos ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow Fin$) pero son linealmente independientes porque cada una ejerce la *arista* S de un nodo predicado distinto, mientras que la Ruta 1 recorre

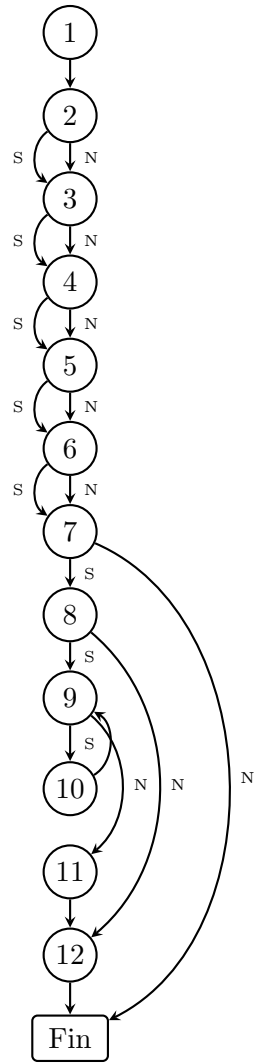


Figura 1: CFG de `liberar_datos`. S = condición verdadera, N = condición falsa.

únicamente aristas N.

1. $1 \rightarrow 2[N] \rightarrow 3[N] \rightarrow 4[N] \rightarrow 5[N] \rightarrow 6[N] \rightarrow 7[N] \rightarrow Fin$
(todos los punteros son NULL)
2. $1 \rightarrow 2[S] \rightarrow 3[N] \rightarrow 4[N] \rightarrow 5[N] \rightarrow 6[N] \rightarrow 7[N] \rightarrow Fin$
(solo **salas** $\neq NULL$)
3. $1 \rightarrow 2[N] \rightarrow 3[S] \rightarrow 4[N] \rightarrow 5[N] \rightarrow 6[N] \rightarrow 7[N] \rightarrow Fin$
(solo **conexiones** $\neq NULL$)
4. $1 \rightarrow 2[N] \rightarrow 3[N] \rightarrow 4[S] \rightarrow 5[N] \rightarrow 6[N] \rightarrow 7[N] \rightarrow Fin$
(solo **objetos** $\neq NULL$)
5. $1 \rightarrow 2[N] \rightarrow 3[N] \rightarrow 4[N] \rightarrow 5[S] \rightarrow 6[N] \rightarrow 7[N] \rightarrow Fin$
(solo **puzzles** $\neq NULL$)
6. $1 \rightarrow 2[N] \rightarrow 3[N] \rightarrow 4[N] \rightarrow 5[N] \rightarrow 6[S] \rightarrow 7[N] \rightarrow Fin$
(solo **jugadores_bd** $\neq NULL$)
7. $1 \rightarrow \dots \rightarrow 7[S] \rightarrow 8[N] \rightarrow 12 \rightarrow Fin$ (**jugador_actual** $\neq NULL$, **inventario** = NULL)
8. $1 \rightarrow \dots \rightarrow 7[S] \rightarrow 8[S] \rightarrow 9[N] \rightarrow 11 \rightarrow 12 \rightarrow Fin$ (**inventario** no nulo, **num_objetos** = 0)
9. $1 \rightarrow \dots \rightarrow 7[S] \rightarrow 8[S] \rightarrow 9[S] \rightarrow 10 \rightarrow 9[N] \rightarrow 11 \rightarrow 12 \rightarrow Fin$ (**inventario** con al menos un objeto)

Casos de prueba — caja blanca:

- CT-1 (Ruta 1)** Todos los campos de juego son NULL. *Salida esperada:* la función termina sin ejecutar ningún **free**; no se produce ningún error.
- CT-2 (Ruta 2)** juego->salas $\neq NULL$; el resto de punteros, NULL. *Salida esperada:* se libera únicamente **salas**.
- CT-3 (Ruta 3)** juego->conexiones $\neq NULL$; salas = NULL. *Salida esperada:* se libera únicamente **conexiones**.
- CT-4 (Ruta 4)** juego->objetos $\neq NULL$; salas y conexiones = NULL. *Salida esperada:* se libera únicamente **objetos**.
- CT-5 (Ruta 5)** juego->puzzles $\neq NULL$; anteriores = NULL. *Salida esperada:* se libera únicamente **puzzles**.

CT-6 (Ruta 6) juego->jugadores_bd $\neq$ NULL; anteriores = NULL.

Salida esperada: se libera únicamente jugadores_bd.

CT-7 (Ruta 7) jugador_actual $\neq$ NULL, inventario = NULL. *Salida*

esperada: se libera jugador_actual; el bloque del inventario no se ejecuta.

CT-8 (Ruta 8) jugador_actual $\neq$ NULL, inventario $\neq$ NULL, num_objetos = 0. *Salida esperada:* el cuerpo del bucle no se ejecuta; se libera

el array de inventario y, después, jugador_actual.

CT-9 (Ruta 9) jugador_actual $\neq$ NULL, inventario $\neq$ NULL, num_objetos $\geq$ 1. *Salida esperada:* se liberan todos los elementos del inventario

de forma iterativa, luego el propio array y finalmente jugador_actual.

Casos de prueba — caja negra:

Estado completamente nulo juego con todos los punteros a NULL.

Salida esperada: la función retorna sin errores ni fugas; Valgrind no reporta accesos inválidos.

Estado completamente inicializado Todos los campos apuntan a memoria válida y el inventario contiene 3 objetos. *Salida esperada:* toda la memoria queda liberada sin *crash*, sin dobles liberaciones ni bloques no liberados.

Inventario con un único objeto num_objetos = 1 con un elemento reservado. *Salida esperada:* el único elemento se libera correctamente antes de liberar el array del inventario; no hay acceso fuera de límites.

Inventario con muchos objetos num_objetos = 50 con cada posición apuntando a memoria válida. *Salida esperada:* los 50 elementos se liberan en orden sin desbordamiento de pila ni acceso a posiciones no reservadas.

jugador_actual válido sin campos de juego Solo jugador_actual $\neq$ NULL; salas, conexiones, objetos, puzles y jugadores_bd = NULL. *Salida esperada:* únicamente la memoria del jugador (e inventario si procede) se libera; los punteros nulos se omiten sin comportamiento indefinido.

4.6. Plan de pruebas de integración

Las pruebas de integración verifican que los distintos módulos del sistema (Salas, Objetos, Puzles y Ficheros) interactúan correctamente cuando se ensamblan en el bucle principal de la partida.

Integración Partida – Puzles – Conexiones

Al resolver un puzle, el sistema debe actualizar el estado del mapa, desbloqueando el acceso a nuevas salas.

Integración Partida – Salas – Inventario

Verifica el trasvase de memoria dinámica al interactuar con los ítems del entorno.

Integración Partida – Ficheros (Guardado y Carga)

Verifica que el módulo de persistencia traduce correctamente las estructuras de RAM a disco y viceversa.

4.7. Plan de pruebas de aceptación

Las pruebas de aceptación validan que el sistema cumple los requisitos del juego recorriendo el flujo narrativo completo con datos reales. Fueron realizadas simulando a un usuario ajeno al equipo de desarrollo, con acceso únicamente al ejecutable y a los ficheros de datos (`.txt`).

ID	Entrada	Salida esperada	Resultado
CI-01	Intentar cruzar conexión bloqueada sin el objeto requerido.	Bloqueo del movimiento. Mensaje narrativo de fallo.	Superado
CI-02	Puzle de puerta resuelto usando el objeto correcto del inventario.	La conexión cambia su estado a “desbloqueada”. El jugador accede a la nueva sala.	Superado

ID	Entrada	Salida esperada	Resultado
CI-03	Acción <code>coger_objeto()</code> sobre un ítem válido de la sala.	El objeto desaparece del array de la sala y se aloja en <code>jugador->inventario</code> .	Superado
CI-04	Consultar inventario tras recoger múltiples objetos.	Se listan todos los objetos sin desbordamiento de memoria ni pérdida de datos.	Superado

ID	Entrada	Salida esperada	Resultado
CI-05	Ejecutar acción de guardado tras cambiar de sala y coger un ítem.	El fichero de guardado refleja el nuevo ID de sala y actualiza el inventario.	Superado
CI-06	Carga de partida con fichero inexistente o corrupto.	Mensaje de error controlado; el sistema retorna al menú principal sin <i>crash</i> .	Superado

ID	Descripción	Entrada	Resultado esperado	OK
PA-01	Registro e inicio de sesión	Credenciales nuevas no presentes en <code>Jugadores.txt</code> .	El sistema registra al usuario y concede acceso al menú principal.	✓
PA-02	Flujo narrativo completo	Partida desde la sala inicial, resolviendo puzles, cogiendo llaves y llegando al final.	Mensaje de victoria, fin del bucle de juego y retorno al menú.	✓
PA-03	Persistencia de la partida	Guardado a mitad de partida; cierre y reinicio del programa; cargar partida.	Reanudación exacta: misma sala, mismo inventario y puertas abiertas.	✓
PA-04	Tolerancia a fallos de entrada	Introducir letras (A, B, C) en menús que esperan números (1, 2, 3).	El sistema limpia el búfer y solicita de nuevo la opción sin colapsar.	✓
PA-05	Acceso a zona restringida	Intentar moverse a una sala con conexión bloqueada sin el objeto necesario.	Mensaje narrativo indicando el bloqueo, sin alterar el estado del jugador.	✓

Cuadro 1: Todos los casos de aceptación se superaron satisfactoriamente.

Referencias